API Security Assessment & Automated VAPT Framework

Comprehensive Technical Guide, Questions & Answers, Practical Labs, and Interview Manual

Target Framework: OWASP API Security Top 10	**Language / Stack:** Python 3.14, Flask, SQLite, Burp Suite
Assessment Status: 16 Baseline Vulnerabilities Found	**Retest Remediation:** 100.0% Verified (0 False Positives)

1. Project Architecture & What Was Built

USER QUESTION: "What did you build in this project, and what are all the components?"

The project is an enterprise-grade **API Vulnerability Assessment and Penetration Testing (VAPT) Framework**. It consists of four distinct engineering systems working in harmony:

1. ShopVault REST API Testbed (api_testbed/)	A realistic e-commerce and document-vault Flask API with SQLite database, multi-user accounts (Alice, Bob, Admin), and a dual security toggle: **Vulnerable Mode** (contains 16 real-world OWASP flaws) vs. **Secure Mode** (hardened production code).
2. Automated Python VAPT Scanner (vapt_framework/)	An automated scanning engine with 8 security modules executing algorithmic attacks against target endpoints. Features a strict **Scope Guard** enforcing testing boundaries.
3. Multi-Format Reporting Engine (reports/)	Automatically aggregates findings and generates an interactive dark-mode **HTML Dashboard**, raw **JSON**, and **Markdown** summaries with CVSS v3.1 scores.
4. Retesting & Verification Suite (retest_engine.py)	A regression engine that diffs baseline scans against hardened scans to mathematically certify patch effectiveness, calculate % remediation rates, and eliminate false positives.

2. Step-by-Step Execution Guide & Windows Commands

USER QUESTION: "Tell me step by step what am I run and see the output result?"

Option A: One-Click End-to-End Automated Demo

Run the all-in-one test suite which automatically starts the API, scans it, restarts in secure mode, and verifies fixes:

```
cd "c:\Users\ASUS\Desktop\API Security Assessment & Automated VAPT Framework"
python run_demo.py
```

Terminal Output: Phase 1 finds 16 vulnerabilities; Phase 2 verifies 0 remaining; Phase 3 certifies 100% remediation rate.

Option B: Two-Terminal Manual Pentest Workflow

Simulating an authentic penetration test requires two Command Prompt windows:

Terminal 1 (The Server)	python -m api_testbed.app --port 5000 --reseed *Keeps the website/API running on port 5000.*
Terminal 2 (The Pentester)	python vapt_cli.py --target http://127.0.0.1:5000 *Runs all 8 modules and finds 16 vulnerabilities.*
Terminal 2 (Burp Suite Mode)	python vapt_cli.py --proxy http://127.0.0.1:8080 *Routes all traffic through Burp Suite proxy.*
Terminal 1 (Fix Mode)	Press Ctrl+C, then run: python -m api_testbed.app --port 5000 --secure
Terminal 2 (Retest & Verify)	python vapt_cli.py --target http://127.0.0.1:5000 --prefix vapt_retest --baseline reports\vapt_scan.json

3. Troubleshooting Common Terminal Errors

USER QUESTION: "I saw errors like 'Start-Process is not recognized', 'WinError 10061', and 'unrecognized arguments'. Why did they happen?"

Error 1: 'Start-Process' is not recognized	**Cause:** Start-Process is a PowerShell command, but you were in standard Windows Command Prompt (cmd.exe). **Solution:** In Command Prompt, use start instead: start reports\vapt_baseline.html
Error 2: [WinError 10061] No connection could be made	**Cause:** The API server on port 5000 was stopped or not running when the scanner ran. **Solution:** Keep Terminal 1 running the API first before launching the scanner in Terminal 2.
Error 3: can't open file 'vapt_cli.py': No such file	**Cause:** The terminal was located in C:\Users\ASUS> instead of the project directory. **Solution:** Run cd "c:\Users\ASUS\Desktop\API Security Assessment & Automated VAPT Framework" first.
Error 4: vapt_cli.py: error: unrecognized arguments	**Cause:** The command was accidentally pasted twice on one line (e.g. ...vapt_scan.jsonpython vapt_cli.py...). **Solution:** Copy and paste the single, clean command.
Error 5: Unable to connect to proxy (port 8080)	**Cause:** --proxy http://127.0.0.1:8080 was specified, but Burp Suite was not opened. **Solution:** Omit --proxy for direct scanning, or open Burp Suite first.

4. Simple Conceptual Explanation (The Bank & The Hacker)

USER QUESTION: "Can you explain simple and short what the project is and why two terminals are used?"

Think of this project using a simple real-world analogy: **A Bank and a Security Guard**.

1. The Bank (Terminal 1 — api_testbed):
This terminal turns ON the web application on your computer (http://127.0.0.1:5000). It stores user accounts, secret corporate files, and balances. In 'Weak Mode', the bank doors have faulty locks.

2. The Security Guard / Penetration Tester (Terminal 2 — vapt_framework):
This terminal launches an automated Python robot that tries 8 different hacking techniques to test if anyone can break into the bank. It proves weaknesses by taking pictures (Proof-of-Concepts) and writing an audit report.

Why 2 Terminals? In real life, you cannot test a bank's security if the bank is closed! Terminal 1 keeps the bank open and alive. Terminal 2 acts as the security auditor testing it.

5. Practical Hands-On Browser Exploitation Labs

USER QUESTION: "Show me in practical how can I understand betterly?"

To understand vulnerabilities with your own eyes, you executed real attacks directly in your web browser:

Lab 1: SQL Injection (Steal All Passwords)	**URL:** http://127.0.0.1:5000/api/v1/products/search?q=' UNION SELECT id, username, password_hash, role, email, is_admin FROM users -- **Result:** The product search failed to validate input, dumping all user accounts and password hashes directly to the screen.
Lab 2: Remote OS Command Injection	**URL:** http://127.0.0.1:5000/api/v1/tools/ping?host=127.0.0.1 & whoami **Result:** The network diagnostic tool passed raw input to the Windows command shell, executing whoami and revealing the computer's username.
Lab 3: Secret Key Disclosure	**URL:** http://127.0.0.1:5000/api/v1/tools/debug/config **Result:** An unauthenticated debug endpoint exposed server secret keys (secret123) and environment paths.
The Security Fix (Hardened Mode)	When restarted with --secure, all three URLs were refreshed: • Lab 1 returned 0 products (safely parameterized queries with ?). • Lab 2 returned 'Invalid host parameter' (regex allowlist block). • Lab 3 returned HTTP 404 Not Found (debug routes completely disabled).

6. Understanding Reports, Remediation & Calculations

USER QUESTION: "Which website was tested? What does remediation mean? How does Python automatically calculate the reports?"

Q: Which website did we test?

You did **NOT** test an outside internet website like Google or Amazon (which is illegal without permission). You tested **ShopVault API** running locally inside your own machine at http://127.0.0.1:5000. In computer networking, 127.0.0.1 is the standard **loopback address (localhost)** meaning 'this computer'. Furthermore, your scanner has a built-in **Ethical Scope Guard** that strictly refuses to scan external URLs.

Q: What does 'Remediation' mean?

Remediation means 'The Security Fix'. Finding a security hole is only half the job; writing the code patch to permanently repair the vulnerability is remediation.

Q: What is the Last Report (retest_verification_report.html)?

In enterprise penetration testing, security teams must prove whether developers actually fixed reported flaws. The Retest Report compares your baseline scan (16 vulnerabilities) against the secure scan (0 vulnerabilities), mathematically certifying a **100.0% Remediation Rate with 0 False Positives**.

Q: How does Python automatically calculate the numbers and build the HTML?

1. **Detection:** When an exploit succeeds, Python appends a Vulnerability object to a list.
2. **Statistics Calculation:** Python calculates Total = len(findings) (16), counts by severity (Critical: 6, High: 7, Medium: 1, Low: 1, Info: 1), and computes:
   Remediation Rate % = (Resolved Findings / Baseline Findings) × 100 = (16 / 16) × 100 = **100.0%**.
3. **HTML Generation:** Python uses the **Jinja2** template engine to inject these values and interactive JavaScript accordions into html_template.py, writing the finished report to reports/vapt_baseline.html.

7. Interview Walkthrough & Presentation Script

USER QUESTION: "If someone says: 'Show me your project and explain it', what should I say and show?"

The 30-Second Elevator Pitch

"This project is an Automated API Security Assessment & VAPT Framework built in Python. I built a dual-mode REST API testbed called ShopVault covering the OWASP API Security Top 10, and an automated multi-module scanner that tests endpoints, generates Proof-of-Concepts, and programmatically retests patches to achieve 100% verified remediation with zero false positives."

The Live 3-Step Screen Demonstration

Step 1: Run Live Demo	Run python run_demo.py in terminal. **What to say:** *"Here the framework launches the API, runs all 8 modules in Phase 1 finding 16 vulnerabilities, restarts in secure mode in Phase 2, and runs the retesting engine in Phase 3."*
Step 2: Show Baseline Report	Run start reports\vapt_baseline.html. **What to say:** Point to **VAPT-BOLA-001** and explain: *"Here, User Bob accessed Alice's confidential M&A; strategy documents by changing ID from 2 to 1 due to missing object authorization."* Then show SQL injection dumping password hashes.
Step 3: Show Retest Report	Run start reports\retest_verification_report.html. **What to say:** *"This is the fix verification audit. It automatically compares the baseline vs. retest scan, confirming that all 16 vulnerabilities were resolved (100% remediation) with zero false positives."*

Top Interview Questions & Winning Answers

Q: How did you test BOLA without high false positives?
*"Simple fuzzing produces high false positives. My framework implements **dual-identity cross-tenant testing**. It authenticates as both a victim (Alice) and an attacker (Bob). It uses Bob's token to request Alice's resource. Only if the response returns HTTP 200 AND contains Alice's private ID and data is a BOLA flaw flagged."*

Q: Can this framework work with Burp Suite?
"Yes. The CLI includes a --proxy http://127.0.0.1:8080 flag that routes all scanner requests through Burp Suite, plus pre-configured raw .http request files for manual validation in Burp Repeater."

8. Resume & Portfolio Entry

Project: API Security Assessment & Automated VAPT Framework
Technologies: Python, Flask, REST APIs, Burp Suite, OWASP API Security Top 10, SQLite, PyJWT
• Engineered an automated API security testing framework in Python with 8 specialized modules covering the OWASP API Security Top 10.
• Developed ShopVault REST API, a dual-mode (Vulnerable vs. Hardened) Flask/SQLite testbed to simulate attack scenarios and remediations.
• Automated dual-token cross-tenant testing to detect Broken Object Level Authorization (BOLA) across private user documents and orders.
• Implemented an automated Retesting & Verification Engine that programmatically compares baseline and post-remediation scans, achieving a 100% verified remediation rate and eliminating false positives.
• Integrated upstream Burp Suite proxy routing and built an interactive HTML reporting dashboard with CVSS v3.1 scoring, wire-level HTTP PoCs, and actionable remediation guidance.